African Cities Experts

GLOBAL PROJECT DOCUMENTATION

Urban Track

The trust layer of urban development in Africa

Cross-verified experience, permanent professional identity and certified CVs

Python 3.12 • Django 6.1 • SQLite • Tailwind CSS • WeasyPrint

Final delivery dossier

Version 1.0 • November 2026

CGa
EMPOWERING INNOVATION, DRIVING GROWTH

Prepared by CGA

March 2026 – October 2026

Document	Global project documentation
Version	1.0: final delivery
Date	November 2026
Prepared by	CGA
Project period	March 2026 – October 2026
Status	Final
Audience	Client, project sponsors, technical evaluators
Product	Urban Track: trust layer for urban development
Repository	`UrbanTrack`: Django modular monolith
Classification	Delivery dossier

Contents

1 Executive summary **1**

2 Context and vision **2**

2.1 The problem . . . 2
2.2 The solution . . . 2
2.3 Value proposition by audience . . . 2
2.4 Guiding business invariants . . . 2

3 Actors, roles and identity **3**

3.1 Account types . . . 3
3.2 The permanent professional identity . . . 3
3.3 Company membership lifecycle (T2) . . . 3

4 Functional scope **4**

4.1 Accounts and identity . . . 4
4.2 Projects . . . 5
4.3 Certification . . . 5
4.4 CV generator . . . 5
4.5 Job board . . . 6
4.6 Administration . . . 6

5 The certification workflow **6**

5.1 States . . . 6
5.2 State machine . . . 6
5.3 Transitions and services . . . 6
5.4 Integrity and edge cases . . . 7

6 Architecture and technical design **8**

6.1 Architectural style . . . 8
6.2 Module dependencies . . . 8
6.3 Request lifecycle . . . 9
6.4 Service layer . . . 9
6.5 Design system . . . 9
6.6 Email infrastructure . . . 10

7 Data model **10**

7.1 Model inventory . . . 10
7.2 Key constraints . . . 11
7.3 Data lifecycle . . . 11

8 Security, privacy and auditability **11**

8.1 Authentication and account protection . . . 11
8.2 Authorization . . . 12
8.3 Data protection and privacy . . . 12
8.4 Application security . . . 12
8.5 Logging and audit trail . . . 13

9 Quality assurance **13**

9.1 Automated test suite . . . 13

9.2 Continuous integration . . . 13
9.3 Testing strategy . . . 14
10 Deployment and operations **14**
10.1 Environments . . . 14
10.2 Production topology . . . 14
10.3 Deployment pipeline . . . 14
10.4 Scheduled tasks . . . 15
10.5 Configuration . . . 15
10.6 Backup and recovery considerations . . . 16
11 Delivery status and roadmap **16**
11.1 Delivered epics . . . 16
11.2 Explicit client decisions . . . 16
11.3 Flagged additive fields . . . 17
11.4 Recommended next steps . . . 17
12 Getting started **17**
12.1 Prerequisites . . . 17
12.2 Local installation . . . 17
12.3 Tests and lint . . . 18
12.4 Project layout . . . 18
A Application route map **18**
B Management commands and scheduled tasks **20**
C Environment variables **20**
D Glossary **21**

1 Executive summary

Urban Track is a trust layer for the African urban development sector. It replaces unverifiable self-declared CVs with **cross-confirmed professional experience**: a company publishes the projects it actually carried out and declares the experts who contributed; each expert personally confirms, adjusts or rejects the declaration. Only experience confirmed by both parties is certified, displayed on public profiles and exported into donor-ready CVs.

The platform is a production-structured **Django 6.1 modular monolith**. The delivered scope covers the complete certification workflow, expert identity and portfolios, project publishing with donor-format metadata, a two-skin bilingual CV generator with PDF and Word export, a job board with application decisions and reminders, a system administration panel, branded transactional email, structured audit logging, and a CI/CD pipeline.

6	22	66	66	288	80%
APPLICATION MODULES	DATA MODELS	APPLICATION ROUTES	HTML TEMPLATES	AUTOMATED TESTS	CI COVERAGE GATE

> **The single message of this delivery.** Urban Track is functional end-to-end today: a user can sign up, obtain a permanent `OX-XXXXXX` identity, publish or join a project, get a contribution independently confirmed, see it certified on a public profile, export it as a professional CV and apply to or publish a mission: with every step traceable and auditable.

Delivered outcomes at a glance

- **Identity that lasts.** Every account receives a free, permanent, unique professional identifier in the `OX-XXXXXX` format, usable as a public URL and on printed CVs.
- **Certification that cannot be self-granted.** A contribution only becomes certified after the named expert personally validates the company's declaration, or after the client counter-signs an independent consultant's role.
- **Certified data that cannot be silently altered.** Once confirmed, the protected fields of a contribution are immutable; any change restarts a complete validation cycle.
- **CVs aligned with donor expectations.** Two professional skins (AFD and World Bank), French and English labels, certified experience visually separated from self-declared sections, and PDF/Word export.
- **A working marketplace.** Job offers, cover letters, accept/decline decisions, notifications and deadline reminders are implemented and tested.
- **Operational maturity.** Environment-driven settings, structured logging with request correlation, audit events, dependency security auditing, automated tests and a VPS deployment workflow.

> **Reading guide.** Sections 2–5 describe the product and its business rules. Sections 6–7 describe how it is built. Sections 8–10 describe how it is secured, tested and operated. Section 11 states exactly what is delivered versus what remains planned, and Section 12 is the developer quick-start.

2 Context and vision

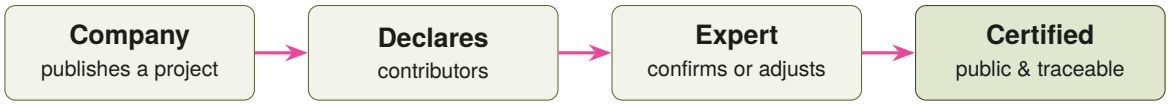
2.1 The problem

In African urban development, hiring and partnership decisions too often rest on CVs that cannot be checked. References are rarely followed up, individual contributors disappear behind institutional signatures, and funders (World Bank, AFD, European Union, national agencies) keep their own siloed records. The consequences are concrete:

- hours lost searching for qualified consultants;
- hiring mistakes caused by unverifiable expertise;
- credit that never reaches the professionals who actually delivered the work.

2.2 The solution

Urban Track applies the co-author logic of academic publishing to the urban sector. The company that carried out a project publishes it and declares its contributors; each contributor is asked to confirm the declaration. The platform is therefore not a CV-hosting website: it is a **verification protocol** with a public, traceable output.



2.3 Value proposition by audience

Audience	Value delivered
Individual experts	A free account, a permanent professional identity, a public profile centred on certified experience, and donor-ready CVs in French and English.
Engineering firms and consultancies	Publish verified references, build certified teams and recruit on traceable track records rather than on claims.
Funders and public institutions	A clearer trust base for partners and suppliers, with declarations traceable to projects and contributors.

2.4 Guiding business invariants

These rules are not guidelines: they are enforced in services, model `save()` methods and database constraints.

1. **Certification comes only from independent cross-confirmation.** A company declares, the expert personally validates.
2. **Adjusting wording re-triggers validation.** A modified contribution must be re-validated by the company.
3. **Existing accounts are linked, never duplicated.** An invited email that matches an account is attached to that account.

4. **Reminders are capped.** At most two automatic reminders, after which the invitation expires (about 14 days).
5. **Humans intervene only for arbitration.** Administrative action is reserved for disputes.
6. **Certified data is immutable.** Protected fields cannot change without a new validation cycle.
7. **Public traceability survives privacy changes.** A private, draft or archived project that holds a certified contribution keeps a minimal public proof page (name, client, dates, certified contributors).

3 Actors, roles and identity

3.1 Account types

The custom user model deliberately separates the *account role* from the *professional relationship* to an organisation.

Account type	Capabilities
User (expert)	The default account. Any user can publish projects, declare contributors, confirm their own contributions, maintain a public portfolio, generate CVs, publish job offers and apply to missions.
Administrator (nominated)	A privileged account nominated by a superuser. It can access the `/admin-panel/` operational console: user management, project and job supervision, application overview.
Superuser	Django's technical administrator. Has full access, including the native Django admin and dispute-arbitration actions.

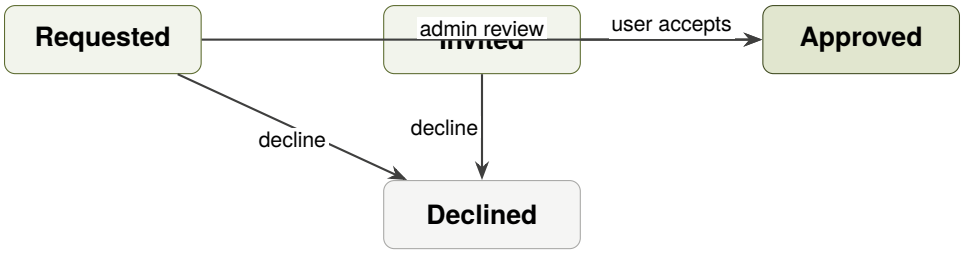
3.2 The permanent professional identity

At first save, every user receives a professional identifier formatted `OX-XXXXXX`. It is:

- **permanent**: generated once and never modified;
- **unique**: enforced by a database unique constraint, with a pre-save collision check and a retry budget;
- **readable**: alphabet stripped of ambiguous characters (`I`, `O`, `0`, `1`);
- **publicly usable**: the expert profile URL is anchored on it, e.g. `/experts/OX-XXXXXX/`.

3.3 Company membership lifecycle (T2)

Joining an existing company is never implicit. The only ways in are: the founder's approved membership at company creation, a request approved by a company administrator, or an administrator invitation accepted by the user. `User.company` is populated only for approved members, so public pages never leak a pending affiliation.



Every review stores `reviewed_by` and `reviewed_at` for audit. The same approved company is what appears on project declarations, public company pages and job offers.

4 Functional scope

The application is divided into six cohesive Django modules plus the project configuration module. Each module owns one business capability and exposes its state transitions through services rather than through ad-hoc view code.

Module	Responsibility
`accounts`	Identity, permanent OX-ID, authentication, email confirmation, company memberships, expert portfolio and public profiles, expert directory.
`projects`	Project model and lifecycle, donor-format referentials, publishing flow, media and reference links, public project directory and proof pages.
`certification`	Contributions, expert invitations, the certification state machine, client counter-signature, reminders, disputes and arbitration.
`cv_generator`	Multi-skin CV context builder, FR/EN labels, PDF (WeasyPrint) and Word (python-docx) exports, CV completeness checks.
`jobs`	Job offers, applications with cover letters, decisions, deadline reminders and the public mission board.
`adminpanel`	System-administration console: operational dashboard, users, projects, jobs and applications.
`main`	Settings, URL routing, middleware, branded email, logging, error handlers and context processors.

4.1 Accounts and identity

- **Sign-up** with a six-digit email confirmation code valid for 24 hours; unconfirmed accounts are purged by a scheduled command.
- **Login** accepts either the email address or the username.
- **Password reset** and **password change** use Django's secure token flows and branded emails.
- **Email change** marks the account unconfirmed and requires a new code before the change is final.
- **Account settings**: personal details, organisation affiliation, avatar.
- **Portfolio editor**: headline, biography, city, country, nationality, phone, birth date (confidential), strengths, countries of intervention, languages, skills, trainings, positions, mandates, publications, teaching, media appearances and free-form miscellaneous content.

- **Dashboard**: published/draft projects, contributions awaiting the user's review, adjusted wordings awaiting company validation, membership approval queue, open job offers, applications and certified track record.
- **Public surfaces**: expert directory with text search, public profile anchored on the OX-ID, public company page, and the public explanation of the trust-score formula.

4.2 Projects

Projects follow a `draft` → `published` → `archived` lifecycle and a `private` or `public` visibility. Publishing is the event that triggers contributor invitations.

Project data	Details
Core	Official name, description, deliverables (one per line), start and end dates, budget, client name.
Donor-format referentials (T6)	Country, funder (distinct from the client), client/master of works, thematic tags, phase (ongoing/completed), intervention volume with unit (person-days/person-months), personal participation window.
Visibility	Draft/published/archived; public/private.
Optional media	Reference links, image gallery, uploaded documents.
Public directory	Full-text search and filters by country, client, funder, phase and tag, paginated.
Proof page	A non-public project with at least one certified contribution exposes a minimal public page so third-party traceability is preserved (T3).

4.3 Certification

The certification module is the heart of the product; it is described in detail in Section 5.

4.4 CV generator

- **Two skins**: *AFD* (consultation format) and *World Bank* (detailed 18-section reference format).
- **Bilingual**: French and English labels loaded from a single shared dictionary (`docs/cv-labels.i18n.json`), with English fallback and logged warnings for missing keys.
- **Builder** at `/cv/` with skin selection, language switch, targeted selection of projects, positions, publications, teaching and media, and a completeness indicator for the World Bank skin.
- **Exports**: inline A4 PDF preview, downloadable PDF through WeasyPrint, and editable Word export through python-docx.
- **Certified versus declared**: certified experience comes first; self-declared sections are rendered under an explicit "declared — not third-party verified" marker and legend (T5).
- **Certified synthesis indicators** (T7): years of experience, certified projects, contributions, countries and donor programmes are computed only from confirmed contributions.
- **Data contract** (T20): each template declares the expected shape of its context; missing or mistyped fields raise explicit errors instead of producing a silently broken CV.
- **Examples**: the idempotent `seed_demo_expert` command creates a fully populated demo expert.

4.5 Job board

- Any authenticated user can publish, edit, close or delete an offer.
- Experts apply with a cover letter; one application per expert per job.
- The publisher reviews applications and accepts or declines them; both decisions trigger a branded notification email.
- Applicants can follow and withdraw their applications; publishers see their offers and received applications.
- A scheduled command sends deadline reminders to applicants and to companies with unreviewed applications.

4.6 Administration

The dedicated system console (`/admin-panel/`) provides an operational dashboard (accounts, projects, jobs, applications, certified contributions, disputes), user search and management, role nomination, account activation/deactivation and deletion, and read-only overviews of projects, jobs and applications. The native Django admin remains available for deep technical operations, including dispute arbitration on certification.

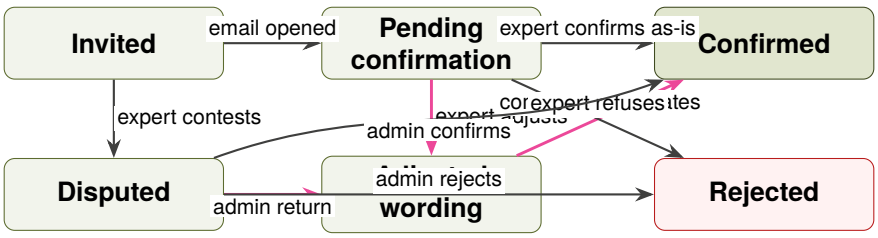
5 The certification workflow

5.1 States

Status	Owner	Meaning
`invited`	Expert	The company declared the contribution; the expert has not acted yet.
`pending_confirmation`	Expert / Company	The expert opened the invitation, or adjusted the wording and awaits company re-validation.
`confirmed`	:	Independently confirmed. The only status that feeds public profiles, trust score and CVs.
`rejected`	:	The expert refused the declaration. Terminal.
`disputed`	Administrator	The expert contests the declaration; arbitration is required.

Invitations carry their own lifecycle: `sent` → `opened` → `converted` or `expired`.

5.2 State machine



5.3 Transitions and services

All transitions live in `certification/services.py` so the rules remain in one auditable place.

Service	Allowed actor	Effect
`declare_contributor`	Project owner	Creates the declaration, auto-links an existing account by email, refuses self-declaration outside the publisher exception.
`notify_contribution`	System	Registered expert: internal notification; unknown email: one active magic-link invitation.
`confirm_as_is`	Named expert	`pending` → `confirmed`; sends a certification email to the company.
`adjust_contribution`	Named expert	Saves the corrected wording, flags company re-validation, notifies the company.
`approve_adjustment`	Declaring company	Confirms the adjusted wording; refused if no adjustment is pending.
`reject_contribution`	Named expert	`rejected`, terminal, reason recorded.
`dispute_contribution`	Named expert	`disputed`; reason is mandatory and the flow freezes until arbitration.
`resolve_dispute`	Staff/superuser	Confirms, rejects or returns to the expert. The only human intervention in the flow.
`request_client_confirmation`	Project owner	Issues a one-time token to the client/master of works for an individual consultant (T1).
`confirm_by_client`	Token holder	Unauthenticated one-time confirmation; the token is revoked on use.
`send_invitation_reminder`	Scheduled task	Sends a reminder, capped at two per invitation.
`expire_stale_invitations`	Scheduled task	Marks overdue invitations `expired` and keeps the audit trail.

5.4 Integrity and edge cases

- **No self-certification.** A physical person cannot be both the declarant and the confirmer of the same contribution. The only exception is the publishing user, whose project-lead contribution is auto-confirmed at publish time.
- **Uniqueness.** A given email can only hold one contribution per project and role.
- **Confirmation timestamp.** A database constraint requires a `confirmed_at` value whenever the status is `confirmed`.
- **Immutability.** On a confirmed contribution, the protected fields (project, expert, role, bullets, invited email) are frozen. Changing one requires leaving the confirmed state, i.e. starting a new cycle.
- **Adjustment loop.** An adjusted contribution returns to `pending_confirmation` with `pending_company_validation=True`; the company cannot certify without an actual expert adjustment.
- **Expiry.** When reminders are exhausted and the expiration date passes, the invitation becomes `expired`; the historical contribution remains visible in the audit trail. The expert can still act on a live invitation through the magic link.
- **Confidentiality.** A private project does not expose its description, deliverables or media on the public proof page; only the certified facts (name, client, dates, contributors) are shown.

Why this design matters. The company can declare, but cannot certify. The expert can certify, but only for themselves. The administrator can arbitrate, but only on a dispute. No single actor can unilaterally create certified experience: which is precisely what makes the badge meaningful.

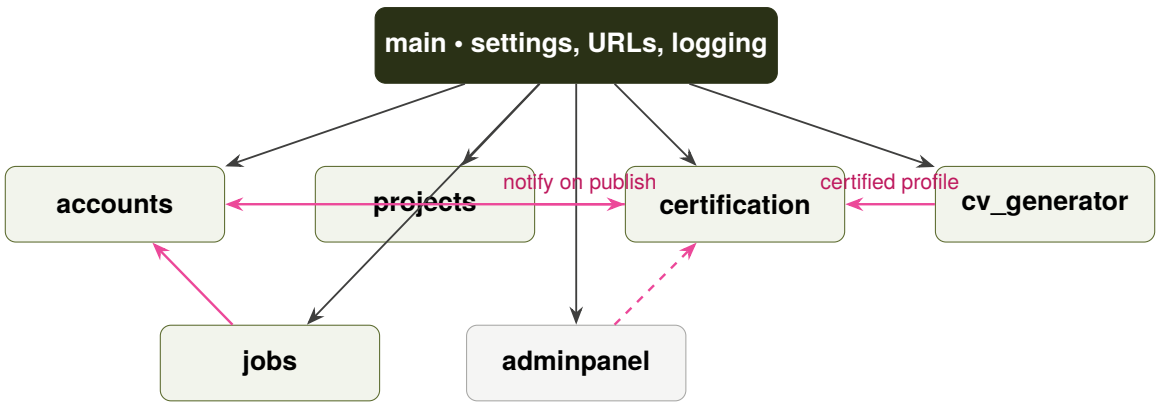
6 Architecture and technical design

6.1 Architectural style

Urban Track is a **modular monolith**: a single Django project composed of cohesive applications with explicit service layers. This choice keeps transactional integrity and operational simplicity while leaving clear seams for future extraction or external connectors.

Concern	Decision
Backend	Python 3.12, Django 6.1, Django REST Framework installed for the API-first intent.
Database	SQLite, by explicit project decision. Strong relational integrity is enforced at the ORM and database-constraint level.
Frontend	Django templates with a vendored Tailwind CSS browser build (pinned), configured inline; no Node/npm pipeline and no external CDN.
Scheduled work	Synchronous management commands, cron-triggerable. No Celery and no Redis, by project decision.
Email	Branded HTML + plain-text emails; console backend in development, SMTP relay (SendGrid) configured through environment variables.
PDF / Word	WeasyPrint for PDF; python-docx for editable Word exports.
Static files	WhiteNoise with a compressed, manifest-hashed storage backend in production.

6.2 Module dependencies



Dependency rule. Dependencies flow through service functions, not through cross-app model manipulation. `projects.services.publish_project` calls `certification.services.notify_contributions_for_project`; `cv_generator` reads confirmed contributions through `ExpertProfile.confirmed_contributions()`. This

> keeps each app testable in isolation.

6.3 Request lifecycle

Every HTTP request passes through the following middleware chain, in order:

1. Django security middleware (headers, HTTPS redirects when configured);
2. WhiteNoise for static files;
3. sessions, common middleware and CSRF protection;
4. authentication and messages;
5. clickjacking protection;
6. `accounts.middleware.RequireEmailConfirmationMiddleware`, which restricts unconfirmed accounts to the confirmation flow;
7. `main.middleware.RequestLogMiddleware`, which assigns a correlation id, logs method/path/status/duration/user/IP, returns the id in the `X-Request-ID` response header and propagates it to every log line.

6.4 Service layer

Business logic is not written in views. The following service modules are the authoritative implementation of the domain rules:

Service module	Responsibilities
`certification.services`	Declarations, invitations, confirmations, adjustments, rejections, disputes, arbitration, reminders, expiry.
`projects.services`	Publishing, archiving, owner contribution, invitation dispatch.
`jobs.services`	Applications, decisions, closing, deadline reminders.
`accounts.email_confirmation`	Code issuance, confirmation, expiry, purge of unconfirmed accounts.
`main.emails`	Absolute URL building and branded email rendering.
`main.logging`	Request correlation, JSON formatting and the `audit()` event helper.

6.5 Design system

The website's visual identity is part of the delivery. Tokens and reusable component classes live in `templates/base.html`:

Token	Value	Usage
`military`	`#5C6B2F`	Headers, navigation, primary buttons, certified badges (dominant colour).
`accent`	`#EC4899`	Calls to action, confirm actions, pending badges, used sparingly.
`charcoal`	`#1C1C1B`	Body text and footer.
Surfaces	`white`, `military-50`	Cards, forms, alternate sections.

Reusable classes include `.btn-primary`, `.btn-accent`, `.btn-outline`, `.card`, `.card-project`, `.card-expert`, `.badge-certified`, `.badge-pending`, `.badge-rejected` and the form system (`.form-label`, `.form-input`, `.form-help`, `.form-error`). The root `color-scheme: light` prevents OS dark mode from inverting form controls.

6.6 Email infrastructure

Every service that sends email goes through `main.emails.send_branded_mail`, which renders a branded HTML template plus a plain-text fallback. Links are always absolute and rooted at `SITE_BASE_URL`. The delivered email templates cover:

- account confirmation code;
- expert invitation, reminder, registered-expert notification;
- company certification notice and adjustment re-validation request;
- client counter-signature request;
- job application received, new application, decision, deadline reminders for experts and companies;
- password reset.

7 Data model

7.1 Model inventory

The delivered system defines **22 models** across four domain groups.

Model	Purpose	Key relationships / notes
Accounts and identity		
`User`	Custom account	Unique email, role, company, avatar, permanent `OX-XXXXXX`, email-confirmation fields.
`Company`	Organisation	Unique name, public profile, membership helpers.
`CompanyMembership`	Affiliation lifecycle	User + company, status, role, requester/reviewer audit trail; unique per pair.
`Skill`	Skill tag	Many-to-many with profiles.
`ExpertProfile`	Public portfolio	One-to-one with user; headline, bio, location, strengths, languages, CV template preference.
`Training`	Education entry	FK to profile.
`Position`	Professional position	FK to profile; declared, not certifying.
`Mandate`	Association mandate	FK to profile.
`Publication`	Publication reference	FK to profile.
`TeachingEntry`	Teaching activity	FK to profile.
`MediaAppearance`	Media appearance	FK to profile; title, outlet, year, URL.
Projects and referentials		
`Country`	Structured country	Unique name and optional ISO-3 code.
`Funder`	Financing institution	Distinct from the client.
`Client`	Client / master of works	Structured referential.

Model	Purpose	Key relationships / notes
`ProjectTag`	Thematic keyword	Shared across projects for filtering.
`Project`	Published project	Owner, lifecycle, visibility, donor-format fields, dates, budget, client; duration constraints.
`ProjectLink`	External reference	Label + URL.
`ProjectMedia`	Optional media	Image or document, caption, upload date.
Certification		
`ProjectContribution`	Declared contribution	Project + expert/email, role, bullets, status, confirmation source, one-time client token; unique project/email/role; confirmed timestamp constraint.
`ExpertInvitation`	Magic-link invitation	Contribution, email, UUID token, expiry, status, reminder count.
Jobs		
`Job`	Job offer	Publisher, location, contract type, description, requirements, compensation, deadline, status.
`JobApplication`	Application	Job + applicant, cover letter, status, timestamps; unique per job/applicant.

7.2 Key constraints

- `User.professional_id` is unique and generated once.
- `User.email` is unique; emails are normalised before save.
- `CompanyMembership` is unique per (user, company).
- `ProjectContribution` is unique per (project, invited email, role), and a confirmed contribution must carry `confirmed_at`.
- `JobApplication` is unique per (job, applicant).
- Project duration must end on or after its start; the personal participation window follows the same rule.

7.3 Data lifecycle

- **Unconfirmed accounts** are deleted after the 24-hour confirmation window by the scheduled purge command (`purge_unconfirmed`).
- **Invitations** expire after about 14 days; reminders are capped at two.
- **Draft projects** can be deleted. Published projects cannot: they must be archived so their certified history survives.
- **Certified contributions** are never deleted through the normal flow; disputes and arbitration are recorded as audit events.

8 Security, privacy and auditability

8.1 Authentication and account protection

- Django authentication with a custom user model; login by email or username.

- Password validators: minimum length 8, similarity check, common-password check and numeric-only check.
- Six-digit email confirmation code, valid 24 hours, required before the account can use the platform (`RequireEmailConfirmationMiddleware`).
- Secure password-reset tokens; password change requires the current password.
- Email changes re-enter the confirmation flow.

8.2 Authorization

- Project management is restricted to the publishing user (or superuser); contribution validation is restricted to the declaring company.
- Contribution actions are restricted to the named expert; sequential primary keys cannot be used to act on someone else's contribution.
- Job decisions and closing are restricted to the offer publisher.
- The system-admin console is restricted to nominated administrators and superusers.
- Company membership approval is restricted to approved company administrators.

8.3 Data protection and privacy

- Personal data is optional and minimised: birth date, phone and nationality are optional; birth date is explicitly kept confidential and never shown on the public site.
- Public profiles display only confirmed contributions as certified experience; self-declared content is labelled as such.
- Private projects expose only a minimal proof page when a certified contribution exists; description, deliverables and media stay private.
- Audit events log identifiers, never secrets or personal data.

8.4 Application security

- CSRF protection on all state-changing forms; clickjacking protection.
- Template auto-escaping prevents HTML injection; the ORM prevents SQL injection.
- Uploaded files are validated by forms and stored under dedicated media paths.
- Secrets and provider credentials are read from environment variables; no secret is committed to the repository.
- `SECURE_PROXY_SSL_HEADER` makes Django trust the reverse proxy's HTTPS indication, so reset and confirmation links use HTTPS.
- One-time UUID tokens protect magic links and client confirmation; the client token is revoked on use.
- Dependency auditing is part of CI (`pip-audit`); recent commits patched known vulnerabilities.

8.5 Logging and audit trail

- Every request is logged with a correlation id returned in `X-Request-ID`; the id is attached to all log lines emitted while handling the request.
- Logs go to stdout (captured by systemd/journald) and, outside tests, to rotating files `logs/urbantrack.log` and `logs/errors.log`.
- `LOG_FORMAT=json` produces one JSON object per line for log collectors; `LOG_LEVEL`, `LOG_DIR`, `LOG_MAX_BYTES`, `LOG_BACKUP_COUNT` and `LOG_SKIP_PATHS` are environment-tunable.
- Business events are written to the `urbantrack.audit` logger: project publishing/archiving, contribution declaration, confirmation, adjustment, rejection, dispute and arbitration, invitation and reminder, membership lifecycle, job application and decision, sign-up, login, email confirmation and account purge.

9 Quality assurance

9.1 Automated test suite

The delivered test suite contains **288 passing tests**, executed in about 26 seconds with an in-memory test database.

Module	Tests	Coverage focus
`accounts`	90	Identity, OX-ID, confirmation, memberships, portfolio, public profiles, settings.
`certification`	64	State machine, invitations, adjustments, disputes, client confirmation, review routes.
`projects`	58	Lifecycle, publishing, referentials, visibility, proof pages, media.
`cv_generator`	34	Context contracts, labels, completeness, exports, demo data.
`jobs`	18	Offers, applications, decisions, reminders.
`adminpanel`	15	Access control and administrative operations.
`main`	9	Logging, middleware, error handlers, email helpers.
Total	**288**	Full platform, including edge cases and permissions.

9.2 Continuous integration

Every push to `main` and every pull request runs three jobs:

1. **Lint**: `ruff check .` and `ruff format --check .`.
2. **Security**: `pip-audit -r requirements.txt`.
3. **Tests**: `manage.py check`, missing-migration detection, static collection, full test suite and `coverage report --fail-under=80`.

> **Quality gate.** A change cannot be delivered if lint fails, a dependency has a known vulnerability, a migration is missing, a test fails, or coverage drops below 80%.

9.3 Testing strategy

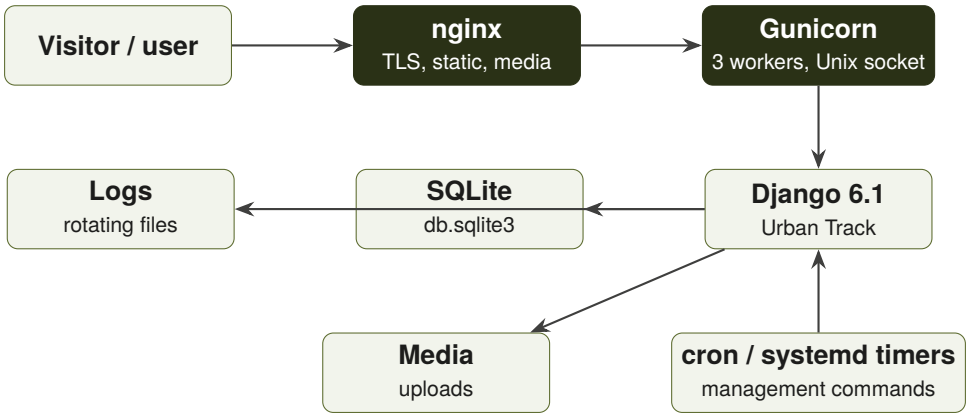
- **Domain-first**: service transitions are tested independently of the views, including invalid transitions and permission failures.
- **Workflow scenarios**: full paths from declaration to certified contribution and from application to decision.
- **Edge cases**: self-declaration, duplicate emails, expired codes, exhausted reminders, withdrawn applications, private proof pages.
- **Contracts**: CV template contexts are validated against explicit data contracts.
- **Presentation**: key pages and access rules are exercised through the Django test client.

10 Deployment and operations

10.1 Environments

Environment	Setup
Development	`runserver`, SQLite, console email backend, local media, rotating logs.
Production	Ubuntu VPS, Gunicorn (3 workers) on a Unix socket, systemd service, nginx reverse proxy with TLS, WhiteNoise for static files, SMTP relay for email, cron for scheduled commands.

10.2 Production topology



10.3 Deployment pipeline

A push to `main` triggers the deploy workflow:

1. checkout and full test run on GitHub Actions;
2. SSH connection to the VPS using a dedicated deploy key;
3. `git fetch origin main` and `git reset --hard origin/main`;

4. virtual-environment self-healing and dependency installation;
5. installation of PDF system libraries (Pango, Cairo, GdkPixbuf);
6. `manage.py migrate` and `manage.py collectstatic`;
7. restart of the `UrbanTrack.service` systemd unit;
8. service health check (`systemctl is-active`).

10.4 Scheduled tasks

Command	Purpose
`process_invitations`	Sends due invitation reminders (capped at two) and expires overdue invitations.
`process_jobs`	Sends deadline reminders to applicants and companies (default horizon: 3 days; `-days` option).
`purge_unconfirmed`	Deletes accounts still unconfirmed after the 24-hour window.
`seed_demo_expert`	Creates an idempotent, fully populated demo expert with certified projects.

Example cron entries:

```
# Certification reminders and expiry
0 8 * * * cd /home/ubuntu/UrbanTrack && env/bin/python manage.py
   process_invitations

# Job deadline reminders
0 9 * * * cd /home/ubuntu/UrbanTrack && env/bin/python manage.py
   process_jobs

# Purge abandoned sign-ups
30 3 * * * cd /home/ubuntu/UrbanTrack && env/bin/python manage.py
   purge_unconfirmed
```

10.5 Configuration

Settings are environment-driven through `.env` (loaded in development) and real environment variables (which always take precedence). Key variables are listed in Appendix C. Notable defaults:

- invitation expiry: 14 days; reminder threshold: 5 days; maximum two reminders;
- confirmation code: 6 digits, 24 hours;
- default email backend: console;
- default logging level: `INFO`, text format;
- static files: WhiteNoise compressed manifest storage;
- database: `db.sqlite3`, configurable via `SQLITE_DB_NAME`.

10.6 Backup and recovery considerations

The production data lives in the SQLite database file and the `media/` directory. A robust operational routine should snapshot both together and keep copies off the VPS. Because SQLite is a single file, backups are simple, but they must be taken consistently (for example with the SQLite online backup API or after a short maintenance stop) and restore procedures should be rehearsed. No health or version endpoint is exposed, by explicit client decision.

11 Delivery status and roadmap

11.1 Delivered epics

Epic	Scope	Status
-1	Bootstrap: repository, applications, SQLite, Tailwind JS, custom user, OX-ID, CI.	DELIVERED
0	Design system and UI foundations: tokens, components, badges, profile typography.	DELIVERED
1	Core data models, cross-confirmation state machine, integrity constraints.	DELIVERED
2	Project publishing screens: create, declare contributors, publish, archive.	DELIVERED
3	Expert invitation and landing: magic link, inline account creation.	DELIVERED
4	Certification and public profiles: directory, project pages, cross-links, trust score.	DELIVERED
5	Edge cases: reminders/expiry command, disputes, admin arbitration.	DELIVERED
6	Email infrastructure: branded templates and console backend. Provider integration and open/click tracking remain.	DELIVERED
7	Multi-template CV generator: AFD and World Bank, FR/EN, PDF/Word, portfolio self-edit.	DELIVERED
8	Job board: publish, apply, accept/decline notifications, deadline reminders.	DELIVERED
9	External connectors: World Bank API, AFD API.	PLANNED
10	AI matchmaking and success prediction.	PLANNED

11.2 Explicit client decisions

The following decisions intentionally override the original generic specification and are part of the delivered architecture:

- **SQLite instead of PostgreSQL**; no database server to operate.
- **No Celery and no Redis**; scheduled work runs as synchronous management commands.
- **Tailwind browser build instead of an npm pipeline**; no Node.js toolchain, no external CDN.

- **No health or versioned endpoints**; operational checks rely on the process manager and logs.
- **SendGrid** retained as the transactional email provider; development uses the console backend.

11.3 Flagged additive fields

None of the mandated data-model fields was altered. The following additive fields were required by the business rules and are documented as deliberate deviations-by-addition:

- `Project.published_by`: ownership of the publishing company user.
- `ProjectContribution.pending_company_validation`: distinguishes “awaiting the expert’s first answer” from “the company must re-validate an adjusted wording”.
- `ProjectContribution.rejection_reason` and `dispute_reason`: audit trail for arbitration.
- `ExpertProfile.cv_template`: per-expert CV skin preference.

11.4 Recommended next steps

1. Connect the SendGrid provider and implement open/click webhooks (completion of Epic 6).
2. Implement the World Bank and AFD connectors (Epic 9).
3. Prepare AI matchmaking on certified data (Epic 10).
4. Add an operational backup and restore runbook for the SQLite database and media directory.
5. Consider PostgreSQL when concurrent write volume justifies it; the ORM layer keeps the migration path straightforward.

12 Getting started

12.1 Prerequisites

- Python 3.12;
- the system libraries required by WeasyPrint (Pango, Cairo, GdkPixbuf);
- on the VPS: Gunicorn, nginx, systemd and a virtual environment.

12.2 Local installation

```
python3.12 -m venv env
source env/bin/activate
pip install -r requirements.txt

cp .env.example .env        # adjust values if needed
python manage.py migrate
python manage.py runserver  # http://127.0.0.1:8000/
```

Create an administrator account, then open `/admin/`:

```
python manage.py createsuperuser
```

Load a fully populated demo expert (idempotent):

```
python manage.py seed_demo_expert
```

Demo credentials: `aminata.sow@demo.urbantrack` / `Demo!Expert2025`.

12.3 Tests and lint

```
python manage.py test          # 288 tests
ruff check .                   # lint (line length 100)
ruff format --check .          # formatting
```

12.4 Project layout

```
main/             Django project: settings, URLs, logging, middleware, email
accounts/         identity, OX-ID, confirmation, memberships, portfolios
projects/         project lifecycle, referentials, media, public proof pages
certification/    contributions, invitations, state machine, arbitration
cv_generator/     multi-skin CV context, FR/EN labels, PDF and Word exports
jobs/             offers, applications, decisions, deadline reminders
adminpanel/       system-administration console
templates/        base shell, design system, public pages, emails
static/           vendored Tailwind build, logos, free-licence photos
docs/             specifications, CV labels dictionary, this documentation
.github/           CI and deployment workflows
```

A Application route map

Path	Name	Access
/	home	Public
/about/	about	Public
/signup/	accounts: signup	Public
/login/	accounts: login	Public
/logout/	accounts: logout	Authenticated
/confirm-email/	accounts: confirm_email	Unconfirmed users
/password-reset/	accounts: password_ reset	Public
/dashboard/	accounts: dashboard	Authenticated
/settings/	accounts: settings	Authenticated
/portfolio/edit/	accounts: profile_edit	Authenticated

Path	Name	Access
/experts/	accounts: expert_list	Public
/experts/<OX-ID>/	accounts: public_ profile	Public
/companies/<id>/	accounts: company_ public	Public
/projects/	projects:list	Public
/projects/new/	projects: create	Authenticated
/projects/<id>/	projects: detail	Public / proof
/projects/<id>/manage/	projects: manage	Owner
/certification/ invitations/<uuid>/	certification: invitation_ landing	Magic link
/certification/ contributions/<id>/ review/	certification: contribution_ review	Named expert
/certification/ client-confirm/<uuid>/	certification: client_ confirm	Token holder
/cv/	cv_generator: builder	Authenticated
/cv/<skin>/	cv_generator: preview	Authenticated
/cv/<skin>/pdf/	cv_generator: pdf	Authenticated
/cv/<skin>/docx/	cv_generator: docx	Authenticated
/jobs/	jobs:list	Public
/jobs/new/	jobs:create	Authenticated
/jobs/<id>/	jobs:detail	Public
/jobs/<id>/apply/	jobs:apply	Authenticated
/jobs/<id>/manage/	jobs:manage	Publisher
/admin-panel/	adminpanel: dashboard	System admin
/admin/	Django admin	Staff
/api/	DRF browsable API	Public surface

B Management commands and scheduled tasks

Command	Behaviour
`python manage.py test`	Runs the full 288-test suite.
`python manage.py migrate`	Applies the 27 migrations.
`python manage.py collectstatic`	Builds the WhiteNoise manifest.
`python manage.py process_invitations`	Reminders and expiry.
`python manage.py process_jobs [-days N]`	Job deadline reminders.
`python manage.py purge_unconfirmed`	Deletes abandoned sign-ups.
`python manage.py seed_demo_expert`	Creates demo data.

C Environment variables

Variable	Default	Purpose
`DJANGO_SECRET_KEY`	dev-only key	Cryptographic signing key; must be set in production.
`DJANGO_ALLOWED_HOSTS`	`localhost,127.0.0.1`	Host header allow-list.
`SQLITE_DB_NAME`	`db.sqlite3`	SQLite database filename.
`SITE_BASE_URL`	`http://localhost:8000`	Absolute base URL used in emails.
`EMAIL_BACKEND`	console	Django email backend.
`EMAIL_HOST`	`smtp.sendgrid.net`	SMTP host.
`EMAIL_PORT`	`587`	SMTP port.
`EMAIL_HOST_USER`	`apikey`	SMTP user (SendGrid).
`EMAIL_HOST_PASSWORD`	empty	SMTP password / API key.
`EMAIL_USE_TLS` / `EMAIL_USE_SSL`	false	Transport security.
`DEFAULT_FROM_EMAIL`	`noreply@urbantrack.africa`	Sender address.
`LOG_LEVEL`	`INFO`	Application log level.
`LOG_FORMAT`	`text`	`text` or `json`.
`LOG_DIR`	`logs`	Log directory.
`LOG_TO_FILE`	`True`	Enable rotating files.
`LOG_MAX_BYTES`	`10485760`	Rotation size.

Variable	Default	Purpose
LOG_BACKUP_COUNT	5	Number of rotated files.
LOG_SKIP_PATHS	static, media, favicon	Paths excluded from access logs.
LAUNCH_COUNTER_THRESHOLD	1	Dashboard launch-stat threshold.

D Glossary

Term	Definition
Certification	Independent cross-confirmation of a contribution by the named expert (or the client for an individual consultant).
Contribution	A declared role and set of achievements of one expert on one project.
Cross-confirmation	The double-validation principle: the company declares, the expert confirms.
Expert invitation	A magic-link email sent to an unregistered contributor, carrying a secret UUID token with an expiry date.
Ox-ID / OX-XXXXXX	The permanent, unique professional identifier of an Urban Track user.
Proof page	A minimal public project page kept accessible when a certified contribution must remain traceable but the project is not public.
Trust score	A weighted sum of certified contributions (director 4, manager 3, specialist/consultant/engineer 2, assistant/other 1).
Declared content	Self-declared portfolio sections (positions, publications, teaching, etc.) that are explicitly not third-party verified.
Skin	A CV visual template: AFD or World Bank.
WeasyPrint	The HTML-to-PDF engine used for CV export.
WhiteNoise	The static-file server used in production without nginx.

End of document

Urban Track • Global project documentation • Final delivery • November 2026